

Sales Pipeline Demo

Technical Report

Vũ Hoàng Tùng

July 23, 2026

Contents

1	Introduction	2
1.1	Resources	2
1.2	Context	2
1.3	Scope and constraints	2
1.4	How I will use AI	2
2	Functional requirements	3
2.1	Roles and access boundaries	3
2.2	Core functions	3
2.3	Role-specific requirements	3
2.3.1	Director	3
2.3.2	Manager	4
2.3.3	Saler	4
3	System architecture and design choices	4
3.1	Design choice	5
3.1.1	Layered architecture	5
3.1.2	Why visitor pattern	5
3.2	Module and submodule mapping	6
4	Data model	7
4.1	Core entities	7
4.2	Relationships and integrity	8
4.3	ETL and data import	8
5	Domain rules	8
5.1	Opportunity stage machine (stage.py)	8
5.2	Reassignment (assign.py)	9
5.3	Permissions (perm.py)	9
6	Recommendation / scoring engine	10
6.1	Visitor pattern for recommendation engine	10
7	Testing	10
8	Conclusions and lessons learned	11
8.1	Assumptions	11
8.2	Achievement and current status vs the requirements	12
8.3	Limitations and future work	14

1 Introduction

1.1 Resources

GitHub repo: <https://github.com/TUng1872004/Sale-app.git>

1.2 Context

Following the requirements, this is a business model under demo: **Sale** → **Agency (Đại lý)** → **Opportunity (Cơ hội bán hàng)**. A sales rep owns a set of agencies; each agency can carry multiple open or closed sales opportunities that need tracking through a pipeline. The brief's functional requirements are intentionally high-level, leaving the author to decide domains, functionality, and implementation details.

It is actually a difficult task for me since I have no experience in sales. So in order to understand the domain, I decided to build a demo around this dataset, which is a real-world dataset of sales opportunities. Dataset. named CRM Sales Opportunities dataset (<https://mavenanalytics.io/data-playground/crm-sales-opportunities>), which ships four CSVs already close to the target schema:

- Table accounts: represent account of customers
- Table products: Products that companies sold
- Table sales_pipeline: shows exactly the spirit of the objectives: a standard sale tracking table tracking employee, customers, stage and revenue
- Table sales_teams: team with members and their managers

Using real, messy third-party data (missing account references, free-text loss reasons, etc.) forces the ETL and domain rules to handle actual edge cases instead of clean synthetic fixtures. See §4.3 for how import gaps were handled.

1.3 Scope and constraints

Given this is a 1 day demo and how unexperienced I am in sales, a lot of things I have here based on naive assumption of the data with a few usecases I can reasoning about. I do not try to anticipate all cases, rather this project aims to show that I can anticipated most cases and propose effective solutions, given more time or actual project planning.

For now the expected scopes are:

- Complete minimal requirements: Sale → Agency → Opportunity, with domain rules and unit tests.
- Document the AI usage and engineering decisions.
- Document the assumptions and trade-offs, the limitations and open questions.
- System Architecture and design options that fit the requirements and business model as good as possible.
- Implement the recommendation engine as an AI feature, with benchmark.

1.4 How I will use AI

The brief explicitly allows AI usage, but asks for it to be documented. I will use Claude to support my reasoning and implementation, with well defined requirements, planning and supervision from me.

The standard workflow of Claude that I use here as well as in all other projects is as follows:

1. Write README.md with requirements and functional specifications. Do it by me
2. Prompt to Claude for skeleton folder structure and module proposal. This will affect the architecture and design of the project, so AI will init standard folders with direct adjustment from me.
3. Modules then stacks. Step 2 creates a solid structure. AI will suggest stacks and implementation with my approval. Also testcases will be generated by AI, decoupled for each module.

4. Craft plan and design for each module. I will prompt to Claude for a plan and design, code then generated after approval.
5. Test and benchmark. I recheck testcases and benchmark generated in previous steps and test core features in each group by hand , then report the result.

Current execution of steps are heavily focused on modularity for further maintenance and fixing, since:

- It is supported by FastAPI well via libraries dedicated to Dependency Injection pattern
- Good for tests, error fix each models without affecting others.
- In that modular blueprint, AI rarely makes mistakes and all are revertable module by module easily with git reset.

2 Functional requirements

This demo addresses the mandatory business chain **Sale** → **Agency** → **Opportunity**. The smallest complete flow is: create a Sale, create an Agency, assign its owner, create an opportunity, and view an overview of the resulting pipeline. The system prioritizes correct relationships, explicit validation, and explainable metrics over the feature breadth of a production CRM.

2.1 Roles and access boundaries

Three roles define the reporting hierarchy: **SALER**, **MANAGER**, and **DIRECTOR**. A Saler works only on their assigned opportunities; a Manager can view and manage work within their team; a Director has global oversight. Minimum authorization rules are implemented as domain functions, rather than relying on a UI to hide prohibited actions.

2.2 Core functions

- Maintain Sales, including role, manager, active status, and ownership relationship.
- Maintain Agencies and record the Sale currently responsible for each Agency.
- Create, update, assign, and delete opportunities linked to exactly one Agency and one owner.
- Move opportunities through an explicit, validated sales pipeline.
- Reassign an Agency while preserving ownership history and revenue attribution for closed opportunities.
- Calculate pipeline value, won revenue, win rate, stage distribution, loss reasons, stale opportunities, and workload.
- Recommend an assignment priority from measurable deal and historical-performance factors.
- Roll back data changes on failed validation or domain-rule rejection, so no partial updates leave the database in an inconsistent state.

2.3 Role-specific requirements

The dashboard explains recommendations directly from data and score factors; no Gemini or other generated narrative is required.

2.3.1 Director

Management tab

- View the overall monthly revenue chart on entry.
- View `manager_logs`, including team-revenue changes and manager reports; each log can open its full report document from shared MinIO storage for this demo.
- View best and worst teams for a selected month, ordered against the standard EMA.

- Drill into a team to view its revenue chart, Saler contribution, and progress.
- Deactivate a Saler directly after recording a reason document. The destructive action uses a three-second hover-to-arm confirmation that turns red only when ready.

Business tab

- Create, view, update, and delete opportunities; select multiple opportunities and assign them to a team.
- Rank opportunity–Saler matches using overall win rate, field-specific Saler win rate, team win rate in that field, field revenue, and average time to win. Factors are normalized dynamically, then reranked by Position as the first-order criterion: a non-matching position is placed below matching positions.

2.3.2 Manager

Management tab

- View the team’s monthly revenue chart, best and worst performers, and comparison with the standard EMA.
- View searchable `salers_logs`, including revenue changes, reports, and requests. Each record can open its full report document from shared MinIO storage.
- Propose, rather than execute, a Saler offboarding action. The same three-second hover-to-arm confirmation applies, and the request is sent to the Director.
- Create and review Frequent Reports about staff, sales details, business issues, or management issues.

2.3.3 Saler

- Update sale details through a `Sale_Report` with all required fields.
- View unhandled opportunities and submit a take-charge request to the Manager.
- Create and submit Frequent Reports about sales, business, or workplace issues.
- Set a seriousness level on Frequent Reports; the level controls the colour and text prominence shown in the superior’s log table.

3 System architecture and design choices

Figure 1 is the layered architecture used for this project. The green *View* block contains authentication, the Jinja2 UI, and FastAPI routes. The blue *Controller* boundary contains two modules: *Service*, which executes workflows requested by the API, and *Domain*, which makes pure business decisions. The orange *Data / Infrastructure* block provides repositories over `SQLModel/SQLite` and a MinIO storage facade.

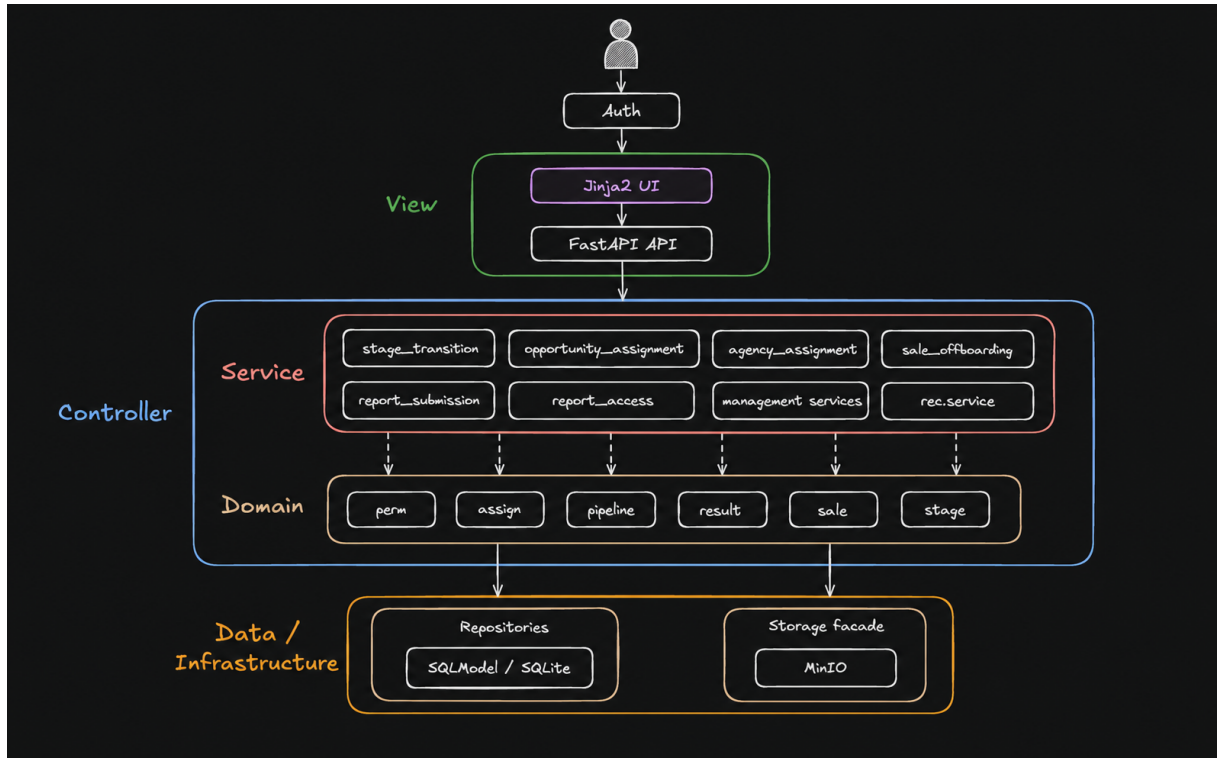


Figure 1: Module architecture. Solid arrows are direct dependencies (import); dashed arrows are dependency injection from Service to Domain.

3.1 Design choice

3.1.1 Layered architecture

As shown in Figure 1, standard operational flow is *controller* $\rightarrow$ *service* $\rightarrow$ *domain*, *repository*, and *infrastructure*. A service may write through the SQLModel session and call the storage facade, while the domain layer must not depend on HTTP, SQLModel sessions, MinIO, or FastAPI. Read-only dashboard pages may call repositories directly because they do not execute a business workflow, but cannot access domain. This decouples the layers, allowing:

- Decouple of FE and BE development, so that FE can be developed in parallel with BE. All FE cares are API endpoints and data format, which are defined in the controller layer. Since schema in core shared module, the core logic are in sync.
- Unit testing of domain rules without a running web server, database, or MinIO instance.
- High level abstraction: service orchestrates domain rules, not repo directly. Requests cannot bypass domain rules, so the business logic is centralized and testable, repos are completely safe.

I utilize FastAPI to inject the current user, database session, and storage facade into controller handlers; configuration injects settings into the database and storage providers. Guarantees that the domain layer is pure and testable without a running web server, database, or MinIO instance. Moreover, the secret information are not copied, only injected into the providers, so the domain layer is also secure. Later use of hashed password, access or refresh tokens and other sensitive information will be safe as well.

3.1.2 Why visitor pattern

Current data from Kaggle: unchanged data, no new data, no new features, no new requirements. Visit pattern decouple logic and data, so new algorithms and data structures can be easily extended by adding new visitors without changing the existing code, or adding any methods.

3.2 Module and submodule mapping

Table 1 follows Figure 1, with `core/` added as a peer source module. The diagram groups the three management files into one *management services* box; the table expands that box to its source files.

Table 1: Architecture modules and submodules.

Module	Module purpose	Submodule	File path	Submodule purpose
View	Handle user interaction.	Auth	<code>app/auth/deps.py</code>	User and role guards.
View	Handle user interaction.	Jinja2 UI	<code>app/templates/*.html</code>	Rendered pages.
View	Handle user interaction.	FastAPI API	<code>app/api/web.py</code>	Routes and responses.
Controller / Service	Run API workflows.	<code>stage_transition</code>	<code>app/services/stage_transition.py</code>	Change stage; write report and log.
Controller / Service	Run API workflows.	<code>opportunity_assignment</code>	<code>app/services/opportunity_assignment.py</code>	Assign individual/bulk deals and requests.
Controller / Service	Run API workflows.	<code>agency_assignment</code>	<code>app/services/agency_assignment.py</code>	Reassign Agencies and log history.
Controller / Service	Run API workflows.	<code>sale_offboarding</code>	<code>app/services/sale_offboarding.py</code>	Execute or decide offboarding.
Controller / Service	Run API workflows.	<code>report_submission</code>	<code>app/services/report_submission.py</code>	Store reports and logs.
Controller / Service	Run API workflows.	<code>report_access</code>	<code>app/services/report_access.py</code>	Authorize report links.
Controller / Service	Run API workflows.	management services	<code>app/services/sale_management.py</code>	Manage Sales and Position.
Controller / Service	Run API workflows.	management services	<code>app/services/agency_management.py</code>	Manage Agencies.
Controller / Service	Run API workflows.	management services	<code>app/services/opportunity_management.py</code>	Create, update, and delete Opportunities.
Controller / Service	Run API workflows.	<code>rec.service</code>	<code>app/rec/service.py</code>	Rank Saler candidates.
Controller / Domain	Apply business rules.	<code>perm</code>	<code>app/domain/perm.py</code>	Check permissions.
Controller / Domain	Apply business rules.	<code>assign</code>	<code>app/domain/assign.py</code>	Decide reassignment behaviour.
Controller / Domain	Apply business rules.	<code>pipeline</code>	<code>app/domain/pipeline.py</code>	Stale-stage thresholds.
Controller / Domain	Apply business rules.	<code>result</code>	<code>app/domain/result.py</code>	Typed rule outcomes.
Controller / Domain	Apply business rules.	<code>sale</code>	<code>app/domain/sale.py</code>	Guard offboarding.
Controller / Domain	Apply business rules.	<code>stage</code>	<code>app/domain/stage.py</code>	Guard stage changes.
Core	Share application configuration and schema.	<code>config</code>	<code>app/core/config.py</code>	Application settings.
Core	Share application configuration and schema.	<code>labels</code>	<code>app/core/schema/labels.py</code>	Display labels.
Core	Share application configuration and schema.	<code>models</code>	<code>app/core/schema/models.py</code>	SQLModel tables and enums.

Module	Module purpose	Submodule	File path	Submodule purpose
Core	Share application configuration and schema.	report	app/core/schema/report.py	Report IDs, documents, and paths.
Data / Infrastructure	Persist data and files.	Repositories	app/repo/*.py	Scoped database reads.
Data / Infrastructure	Persist data and files.	SQLModel / SQLite	app/db.py	Engine and sessions.
Data / Infrastructure	Persist data and files.	Storage facade	app/storage.py	Storage abstraction.
Data / Infrastructure	Persist data and files.	MinIO	Docker Compose service	Report documents.

The table concludes the architecture and design choices, laying solid foundation to later implementation. The next section describes the domain rules that enforce the business logic.

4 Data model

Six tables implement the Sale → Agency → Opportunity chain, assignment history, and separate manager/Saler audit logs. Defined in `app/core/schema/models.py` as `SQLModel` classes (single source of truth for both the Python types and the SQL schema, migrated via Alembic).

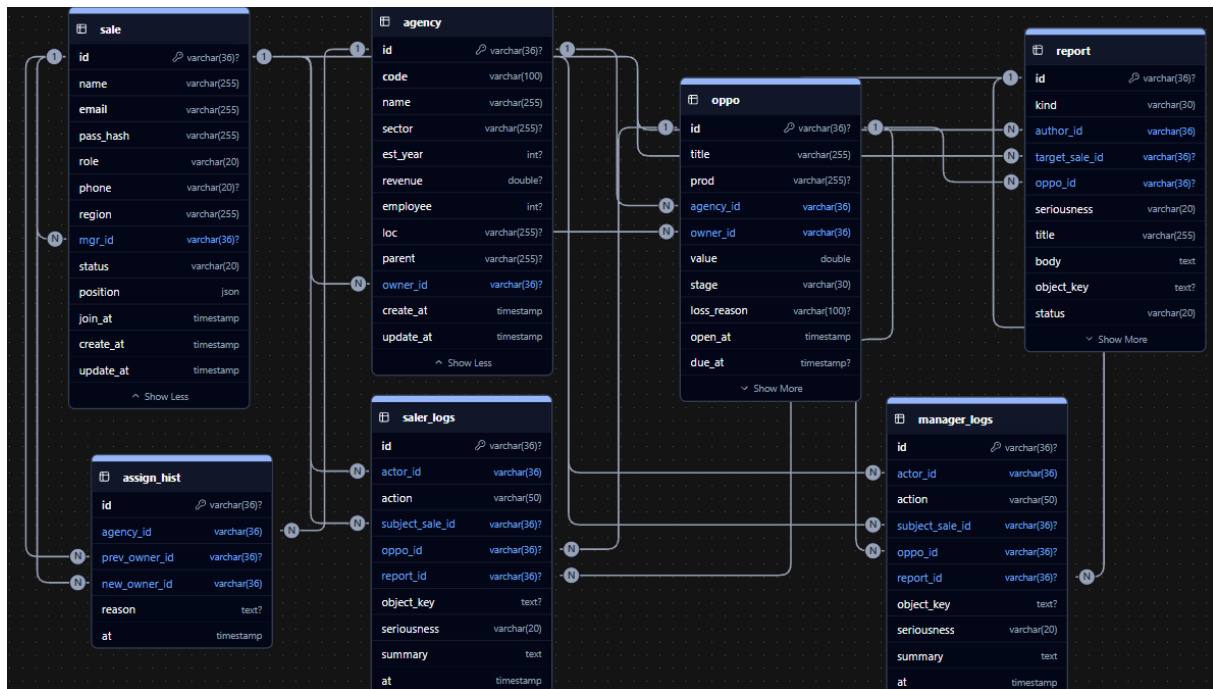


Figure 2: Database schema.

4.1 Core entities

Sale A user: Saler, Manager, or Director (`role` enum). Self-referential `mgr_id` builds the reporting tree (null for Director). Carries `status` (ACTIVE/INACTIVE) rather than deleting the row on offboarding — see §8.1.

Agency An account/customer. `owner_id` points at the responsible Sale (nullable: an agency can be temporarily unowned, e.g. between reassignments).

Oppo An opportunity tied to exactly one Agency and one owning Sale. Carries `stage`, optional `loss_reason`, and timestamps (`open_at`, `due_at`, `close_at`) that drive the staleness and scoring logic in §6.

AssignHist Audit trail of ownership changes on an agency: previous owner, new owner, free-text reason, times-tamp. Exists so "who owned this and why did it move" is answerable without inferring it from side effects.

4.2 Relationships and integrity

- `Sale.mgr_id` → `Sale.id` (self-FK, nullable) — reporting line.
- `Agency.owner_id` → `Sale.id` (nullable) — current owner.
- `Oppo.agency_id` → `Agency.id` (required) — every opportunity belongs to exactly one agency.
- `Oppo.owner_id` → `Sale.id` (required) — every opportunity has an explicit owner, independent of the agency's current owner (see §8.1 on reassignment semantics).
- A composite index on (`stage`, `open_at`) backs the stale-opportunity query (filter to open stages, then compare age) — added after the query pattern was known, not speculatively.

4.3 ETL and data import

`app/seed.py` imports the four Maven Analytics CSVs into the schema above. The import is idempotent (safe to re-run) and skips rows that cannot map cleanly onto the schema:

- 1,425 of 8,800 source `sales_pipeline` rows had a blank `account` field and were skipped — verified by direct inspection of the raw CSV that every skipped row is an open-stage deal, so closed-deal counts (4,238 Won / 2,473 Lost) are unaffected by the skip.
- Expected seed counts were corrected mid-implementation once the real 16% skip rate was discovered, rather than leaving a stale target a future test run would fail against for a non-bug reason.

5 Domain rules

Business logic lives in `app/domain/` as pure functions with zero I/O — no ORM session, no HTTP, so every branch is directly unit-testable and independent of the framework underneath it. Each function returns a `Result` (Ok/Err) rather than raising, so callers get a typed reason code instead of parsing an exception message.

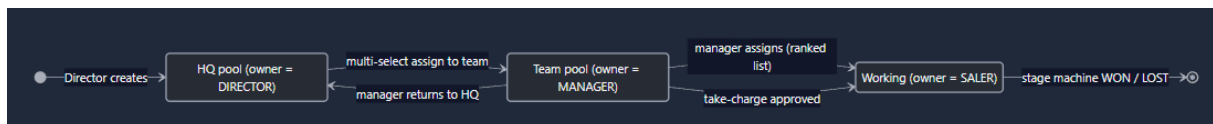


Figure 3: Opportunity ownership lifecycle: From Director to Manager to Salers.

5.1 Opportunity stage machine (`stage.py`)

Normal sales states are `NEW` → `QUALIFY` → `PROPOSE` → `NEGOTIATE` → `{WON, LOST}`. `CLOSED` is an additional terminal state used only when a Director deletes an open Opportunity that already has audit history. Rules enforced by `can_transition`:

- Forward-only — a deal cannot move backward in the pipeline or skip a stage; both are rejected with distinct error codes (`BACKWARD`, `SKIP_STAGE`).
- `WON`, `LOST`, and `CLOSED` are terminal — no further transition once closed (`TERMINAL`).
- Marking `LOST` requires a `loss_reason`; marking `WON` rejects one being present. This trades a small amount of flexibility for data that is always explainable when a deal is lost.

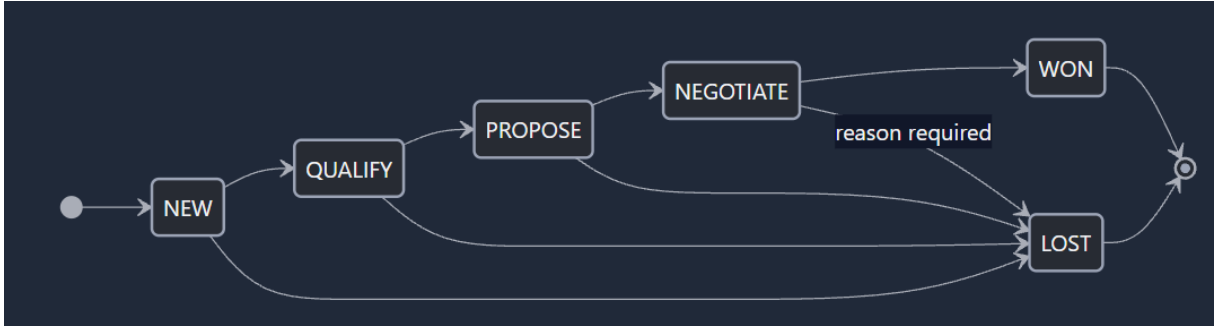


Figure 4: Opportunity stage transitions.

5.2 Reassignment (assign.py)

An agency's owner can change (`can_reassign`) only to an ACTIVE Sale. Reassigning an agency does *not* blanket-move every historical opportunity: closed opportunities (WON/LOST) stay credited to the original owner for revenue-attribution integrity; open opportunities move with the agency to the new owner (`oppo_follows_reassign`). This distinction was a deliberate answer to one of the brief's open questions (§8.1).

5.3 Permissions (perm.py)

One function per capability, checked in application code before any mutation — not left to implicit role checks scattered across routes:

Capability	Rule
View an opportunity	Owner, owner's manager, or any Director
Reassign within a team	Manager of the agency's current owner, or Director
Reassign across teams	Director only
Deactivate a Sale	That Sale's manager, or Director
Export global data	Director only

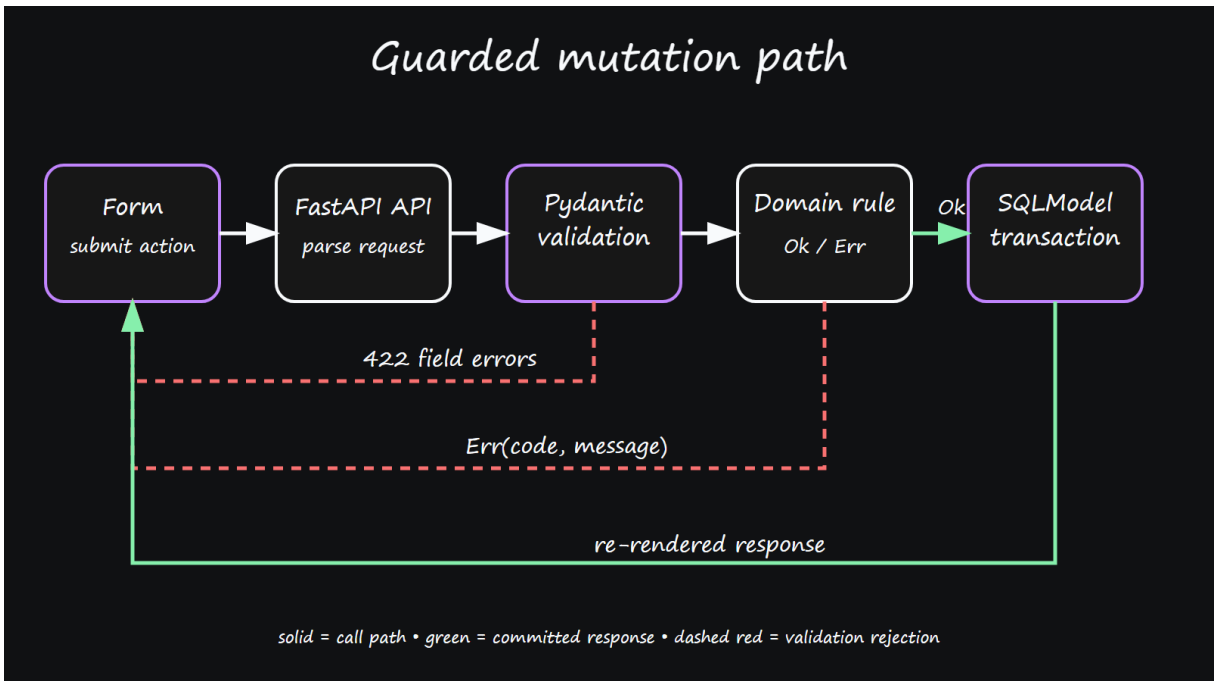


Figure 5: Guarded mutation path: validation and domain failures return without writing data; an accepted action commits in a transaction.

6 Recommendation / scoring engine

The Director-facing “recommend opportunities to salers” feature (per README.md’s functional requirements) is implemented as a small scoring pipeline under `app/rec/`, structurally a functional Visitor: each factor is an independent function over a shared `DealCtx`, and `registry.py` folds their outputs into one priority score — extending the model means adding one file to the VISITORS list, not editing a growing branch tree.

Factors (`rec/visitors/`): stage weight, staleness (days in stage past a per-stage threshold, capped impact), deal value vs. agency mean (outlier penalty), saler’s current load, and historical win-rate in that field/team. Weights and caps are centralized as frozen constants in `rec/weight.py` rather than scattered literals, so tuning is a one-file change.

6.1 Visitor pattern for recommendation engine

The opportunity-priority scorer applies a *functional Visitor* pattern. Instead of a classic object-oriented double-dispatch design, `DealCtx` is the element being visited and each function in `app/rec/visitors/` implements the same callable protocol: `DealCtx → Factor | None`. The functions are registered in the ordered VISITORS list in `registry.py`; the registry invokes every visitor, keeps the non-empty factors, and sums their impacts into a `Score`.

This design keeps visitors pure: the caller prepares a complete `DealCtx`, so visitor functions perform no database or network I/O. Adding a scoring factor therefore means adding one visitor file and registering it, without changing the existing factor implementations. The resulting `Factor` objects retain a signal name and Vietnamese display explanation, so a recommendation can be explained from its component scores rather than from generated narrative.

The candidate-Saler recommender in `rec/service.py` is a separate workflow. Its learned model is a binomial Generalized Linear Model (GLM), not ordinary linear regression: the model uses standardized overall win rate, field-specific win rate, team field win rate, field revenue, and average days to win. The benchmark artifact stores the scaler, GLM coefficients, and its selection result; runtime retains a deterministic fallback if that artifact is invalid. Both mechanisms remain explainable, but the functional Visitor pattern specifically describes the opportunity-priority pipeline in `rec/registry.py`.

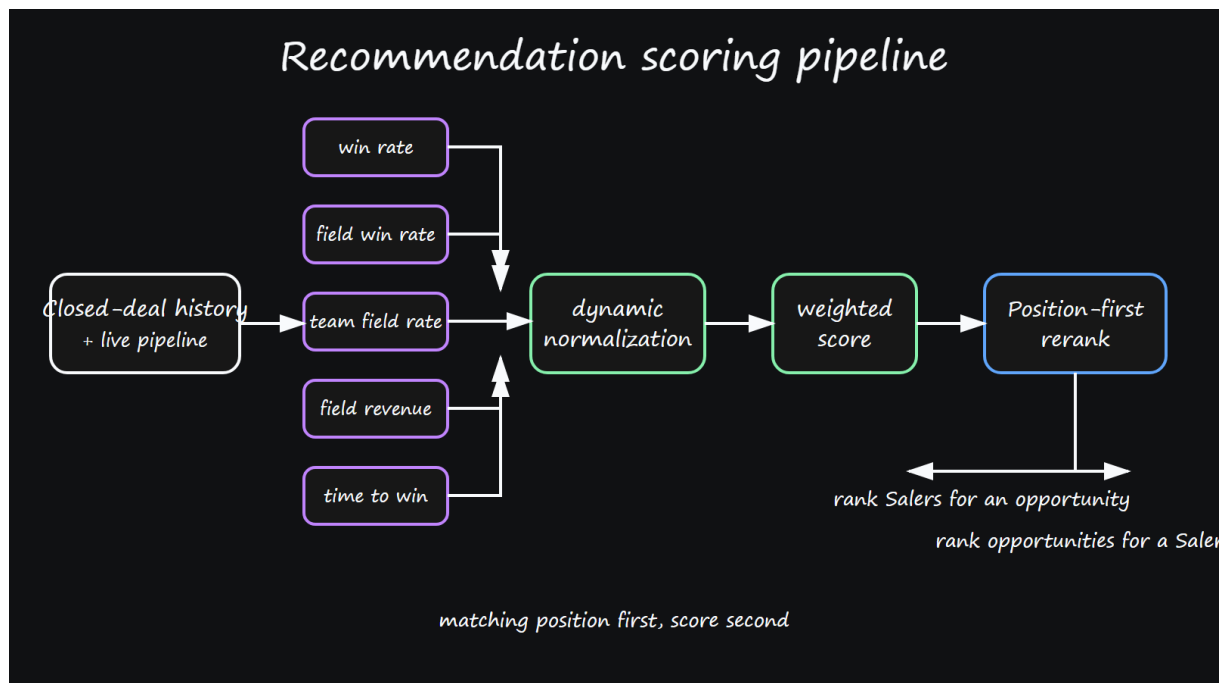


Figure 6: Recommendation scoring pipeline with dynamic normalization and Position-first reranking.

7 Testing

The suite contains 42 representative pytest cases, all passing at the time of writing. It selects the decision boundaries and end-to-end flows that demonstrate each delivery phase, rather than repeating every equivalent input per-

mutation. The parameterised Lost-stage test accounts for four of the 42 cases.

Table 2: Representative automated tests by implementation phase.

Phase	Test file	What is tested	Cases
Phase 1: domain and access	test_stage.py	test_negotiate_to_won_allowed, no skipped or backward stage, four open-stage-to-Lost paths, terminal Won/CLOSED, and required loss reason.	10
Phase 1: domain and access	test_assign.py	Active/inactive reassignment target; open opportunity follows a reassignment while Won and Lost opportunities do not.	4
Phase 1: domain and access	test_sale.py	Offboarding is rejected for a Sale owning agencies, an inactive Sale, or a Sale owning an open opportunity.	3
Phase 1: domain and access	test_perm.py	Saler self/other scope, Manager own/other-team scope and team reassignment, plus Director cross-team reassignment.	6
Phase 2: data and ETL	test_repo.py	Scoped list pagination, team membership, KPIs, stage and loss summaries, workload order, completed-month revenue, EMA, rankings, and stale boundary.	3
Phase 2: data and ETL	test_seed.py	Deterministic agency-owner tie-break; initial seed, idempotent verification, reseed, and required Saler Position data.	2
Phase 3: workflows and HTTP	test_storage_workflow.py	Upload/transaction compensation: SQL failure removes the uploaded document; upload failure makes no SQL write.	2
Phase 3: workflows and HTTP	test_http.py	Login redirect and scope denial; Director CRUD, bulk assignment, history-protecting deletion, and offboarding guard; stage report and log; team-pool request/approval; dismissal reason and manager report storage.	7
Phase 4: recommendation	test_benchmark.py	Exact Recall/Precision@3 and @5 calculation; Position match ranks ahead of a higher-scoring non-match.	2
Phase 4: recommendation	test_registry.py	Won/Lost opportunities receive zero priority; open opportunities rank in descending priority.	2
Phase 4: recommendation	test_rec_service.py	Invalid or incompatible benchmark artifact falls back to the deterministic recommender.	1
Total			42

The passing run provides repeatable verification evidence for the listed flows. mypy runs clean across app/ and tests/ with a single documented, justified suppression on a test helper’s flexible kwargs (not production code).

8 Conclusions and lessons learned

8.1 Assumptions

The brief explicitly asks these to be asked and documented. The following are best-effort guesses based on the data, the business domain, and the requirements. They are not guaranteed to be correct, but they are consistent with the:

Sale nghỉ việc (offboarding) — how is old data handled?

A Sale is marked INACTIVE, never deleted. Historical opportunities and agencies keep their original owner_id for reporting integrity; new work cannot be assigned to an inactive Sale (can_reassign rejects it). Limitation: no automated ”bulk reassign all of this Sale’s open work” flow exists yet — a manager or director must reassign agencies individually.

Can an agency be transferred to another Sale?

Yes, within a manager’s own team by that manager, or across teams by a Director only (§5). Closed opportunities stay with their original owner; open ones move with the agency (assign.py). Trade-off: this optimizes for correct revenue attribution over a simpler ”everything moves” rule.

What states can an opportunity be in, and how does it transition?

Six stages, forward-only, two terminal states, loss reason mandatory on loss — detailed in §5.1.

Are duplicates allowed? Can created data be deleted?

Not yet enforced at the domain layer; the demo currently trusts the seed ETL's dedup logic (agency code is a unique slug of the account name) and does not expose hard deletes through the web layer — soft-deactivation (status) is the only lifecycle-ending operation modeled, deliberately, so history is never destroyed by a UI action.

Who can view or change data?

Three-role hierarchy (Saler / Manager / Director), enforced by the permission functions in §6's domain-rules section, not by decorative role flags.

How are overview metrics computed?

Dashboard aggregates (pipeline value, won revenue, win rate, stage distribution, loss reasons, stale-opportunity count, per-saler workload) are computed in `app/repo/stats.py` directly from `Opportunity` rows scoped by owner IDs, so the same formulas serve saler, manager, and director views without duplication. Exposed through the dashboard views described in §8.2.

8.2 Achievement and current status vs the requirements

Layer	Status
Schema & migrations (Alembic)	Done
Seed / ETL from source CSVs	Done, idempotent
Domain rules (stage, assign, perm)	Done, unit-tested
Repository / query layer	Done, unit-tested
Recommendation / scoring engine	Done, unit-tested
HTTP routes (<code>app/api/</code>)	Done
Auth (<code>app/auth/</code>)	Done
Templates / UI (<code>app/templates/</code>)	Done
Opportunity CRUD and bulk assignment	Done, HTTP-tested

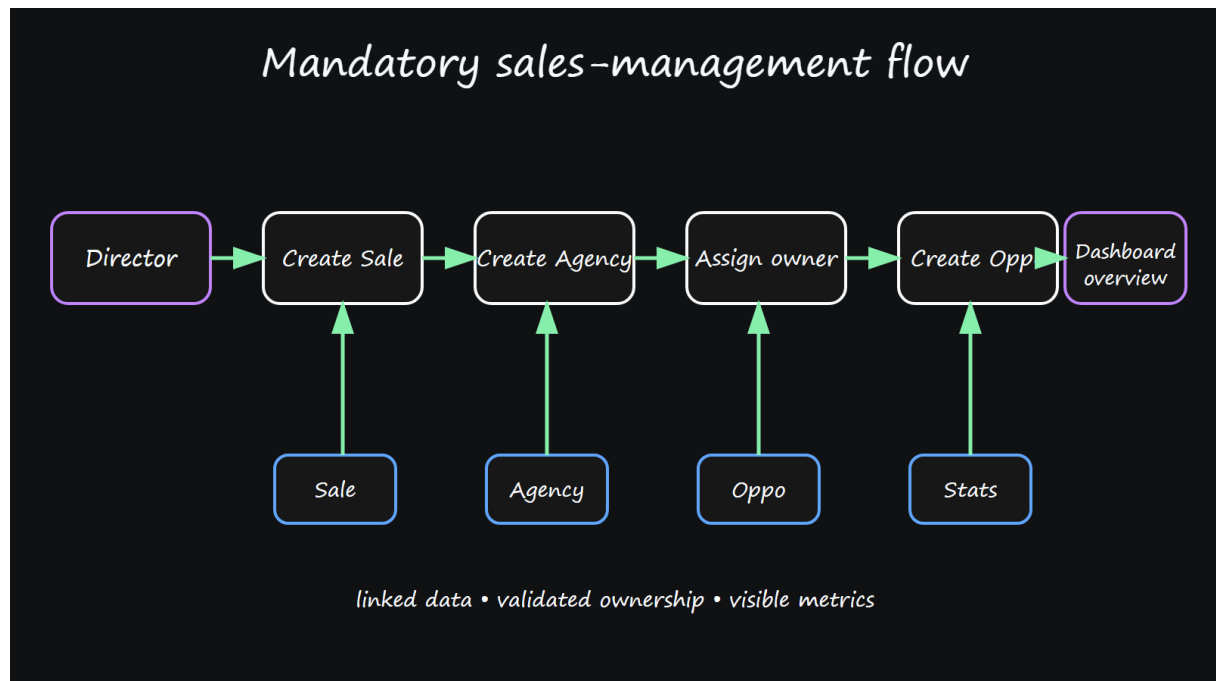


Figure 7: Mandatory Sales → Agency → Opportunity flow and the data it creates.

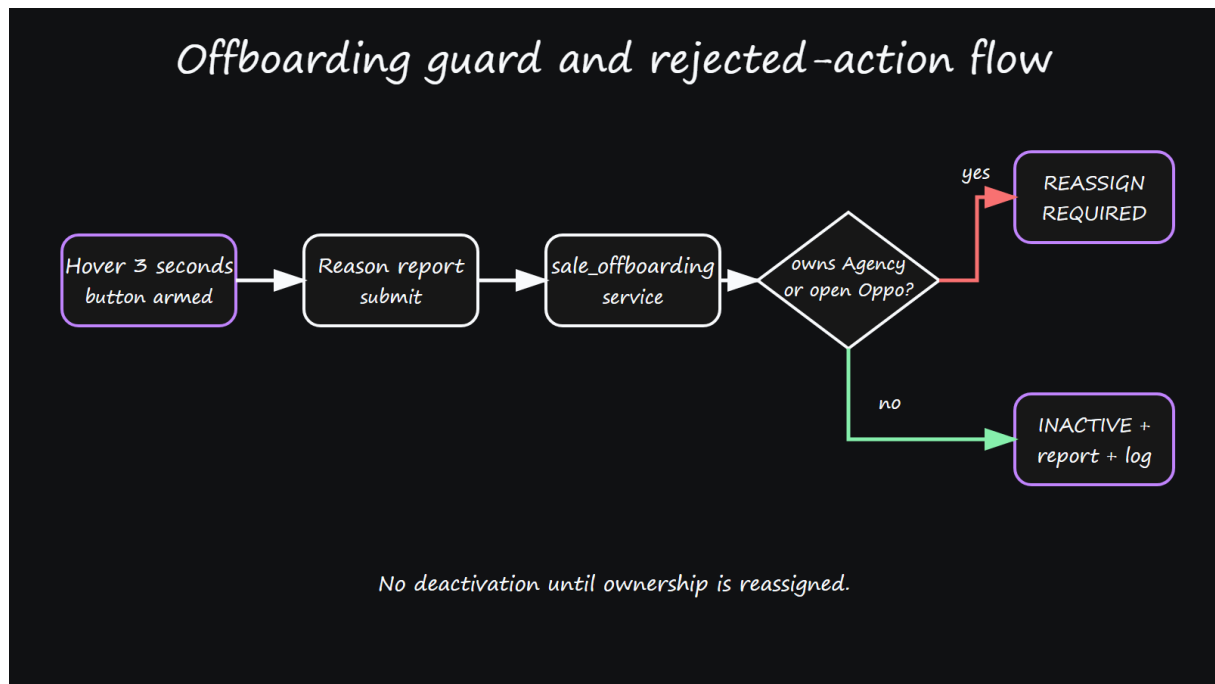


Figure 8: Rejected offboarding flow: ownership must be reassigned before a Sale can become inactive.

The full *Create Sale* → *Create Agency* → *Assign* → *Create opportunity* → *View overview* flow is implemented end-to-end through the server-rendered FastAPI interface. Routes receive the current user, database session, and storage facade through dependency injection; write routes delegate to application services, while read views use the shared repositories and dashboard statistics.

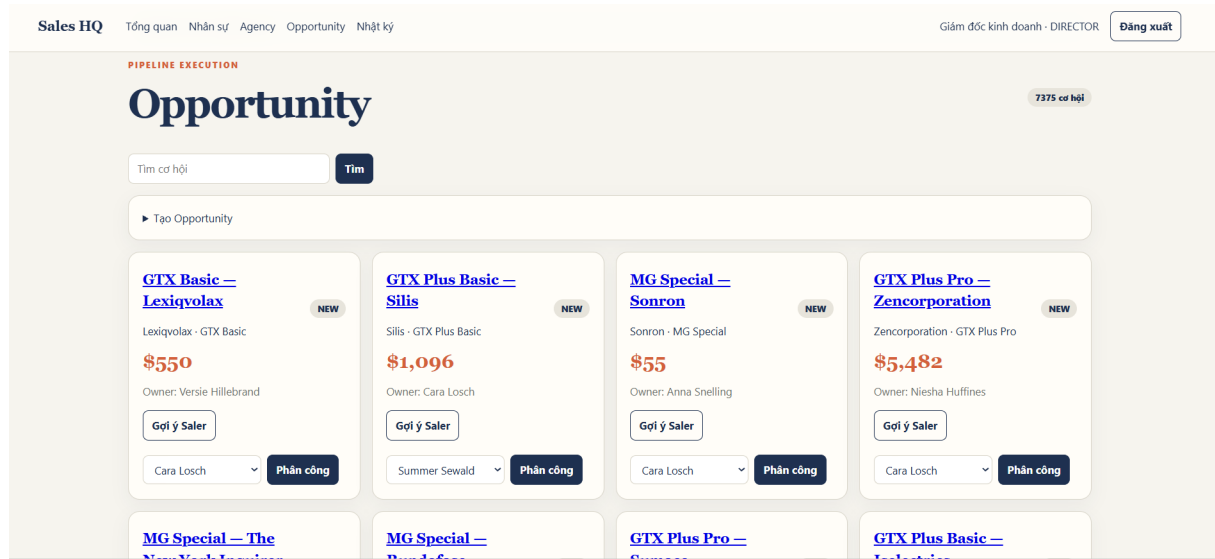


Figure 9: Director Opportunity Overview.



Figure 10: Director Revenue Chart.

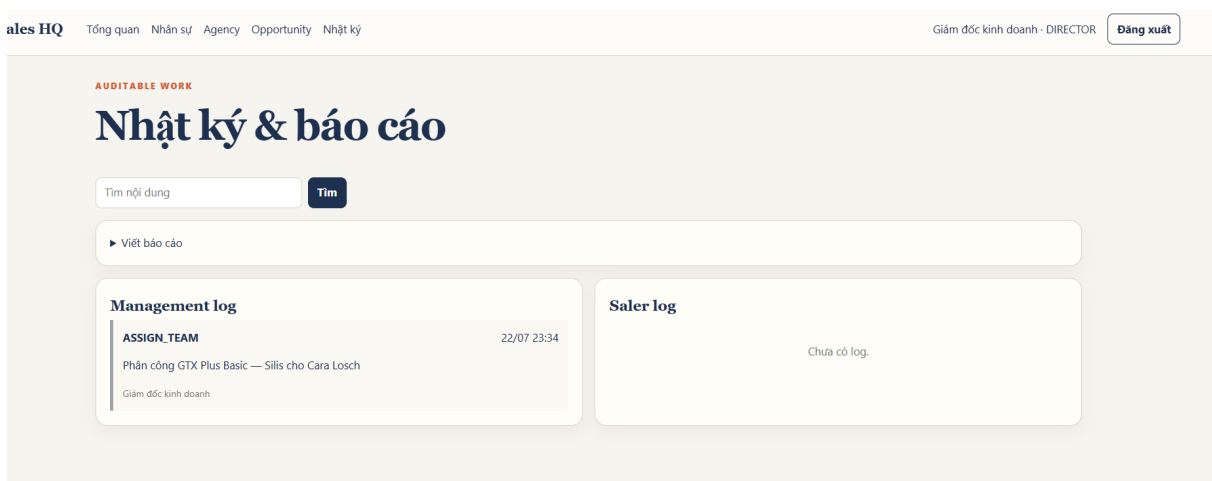


Figure 11: Director Revenue Log.

8.3 Limitations and future work

The following are the main points that need to be improved in the future:

- No full browser end-to-end records yet, only technical documentation.
- No duplicate-detection semantics at the domain layer. A Director can hard-delete an open Opportunity without audit history; once history exists, deletion marks it CLOSED instead, preserving evidence.
- No automated bulk-reassignment on Sale offboarding.
- Domain modeling reflects best-effort assumptions from an data and business domain (sales operations), not exhaustive real-world coverage — by design, per §8.1. Need more time to research and understand the domain, then adjust the model and rules accordingly.
- No production-grade security, hardening, logging, or monitoring. All is in a demo state, with the exception of the domain rules and permission checks which is about logic more than real infrastructure.
- The binomial GLM is trained from historical CSV outcomes only. More live outcomes, richer business features, and periodic retraining are needed before treating its ranking as production-quality prediction.

- AI can be used for autonomous sales management, but it is not implemented yet, since data with such short field body make GenAI a trivial task. If there are more complex case with more data and more complex business rules, cheap AI integration can be studied.